Compte rendu détaillé

Audit de sécurité et durcissement de la bibliothèque AirLLM

Référentiel : main @ ac9e260 · Périmètre : package air_llm · Date : 14/10/2026

Sommaire

Le présent document rend compte de l'audit de sécurité mené sur la bibliothèque AirLLM, des vulnérabilités identifiées, des mesures de durcissement appliquées, de leur validation par les tests, et de l'état résiduel. Il est structuré en treize sections détaillées et quatre annexes.

§	Section
1	Contexte et objet de l'audit
2	Périmètre et méthodologie
3	Synthèse des risques (cartographie)
4	V1 — Exécution de code arbitraire via pickle (critique)
5	V2 — Exécution de code distant non maîtrisée (élevé)
6	V3 — Robustesse et suppression de fichiers (moyen)
7	V4 — Intégrité de la chaîne d'approvisionnement (moyen)
8	Phase 2 — Généralisation tri-état de trust_remote_code
9	Construction des impacts : matrice avant / après
10	Volumétrie des changements
11	Stratégie de test et validation
12	Risques résiduels et limites assumées
13	Conclusion et recommandations
A	Annexe A — Détail des commits
B	Annexe B — Inventaire des fichiers audités
C	Annexe C — Références des correctifs (extraits de code)
D	Annexe D — Glossaire

1. Contexte et objet de l'audit

AirLLM est une bibliothèque d'inférence de modèles de langage (LLM) qui permet d'exécuter des modèles de très grande taille — jusqu'à 2 800 milliards de paramètres (Kimi K3) — sur des cartes graphiques à mémoire limitée, grâce à une technique de streaming des poids couche par couche depuis le disque. Elle prend en charge l'inférence et, depuis peu, l'entraînement (LoRA) de modèles massifs sur de petites cartes.

Ce mécanisme implique par nature la manipulation d'artefacts non fiables : des fichiers de poids de modèles téléchargés depuis des dépôts tiers (HuggingFace), du code Python « distant » embarqué dans ces dépôts, et de nombreuses opérations d'entrée/sortie sur le système de fichiers (téléchargement, découpage, réécriture, suppression). Ce sont précisément les surfaces d'attaque classiques d'une telle bibliothèque.

L'objet de cet audit est d'identifier, de qualifier et de corriger les vulnérabilités de sécurité présentes dans le package air_llm, en priorité celles pouvant conduire à une exécution de code arbitraire sur la machine de l'utilisateur, sans dégrader le fonctionnement nominal de la bibliothèque.

2. Périmètre et méthodologie

2.1 Périmètre

L'audit a couvert l'intégralité des 46 fichiers Python du dépôt, avec une attention particulière portée au package air_llm/airllm (le cœur de la bibliothèque). Les fichiers périmètre secondaire — scripts d'entraînement (training/, rlhf/), jeux de données et notebooks — ont été examinés pour leur usage des mêmes primitives sensibles.

2.2 Méthode

L'analyse repose sur un examen statique du code source et la recherche systématique de motifs à risque, regroupés en quatre familles de menaces :

- **Désérialisation non sûre —** torch.load() sans weights_only, pickle, marshal.
- **Exécution de code distant —** trust_remote_code=True non contrôlable.
- **Opérations sur le système de fichiers —** suppression, liens symboliques, traversée de chemins.
- **Chaîne d'approvisionnement —** dépendances épinglées sur des références mouvantes.

Chaque risque est qualifié selon une échelle de gravité à quatre niveaux : Critique, Élevé, Moyen, Faible. La criticité reflète l'impact potentiel sur les systèmes exécutant la bibliothèque (compromission complète de la machine hôte pour une exécution de code arbitraire).

3. Synthèse des risques (cartographie)

L'audit a mis en évidence quatre risques distincts, tous corrigés. Le tableau suivant les synthétise ; chaque risque fait l'objet d'une section dédiée (4 à 7).

Réf.	Gravité	Vecteur d'attaque	Impact potentiel	Statut
V1	CRITIQUE	torch.load() sans weights_only	Exécution de code arbitraire (RCE)	Corrigé
V2	ÉLEVÉ	trust_remote_code=True codé en dur	Exécution de code distant non maîtrisée	Corrigé

Réf.	Gravité	Vecteur d'attaque	Impact potentiel	Statut
V3	MOYEN	remove_real_and_linked_file()	Plantage (UnboundLocalError) ; suppression non gérée	Corrigé — chaîne validée
V4	MOYEN	Dépendances git non épinglées	Builds non reproductibles ; compromission supply chain	Corrigé

Aucune vulnérabilité Critique ou Élevée ne subsiste après application des correctifs. Le détail de chaque mesure figure en sections 4 à 8.

4. V1 — Exécution de code arbitraire via pickle (CRITIQUE)

4.1 Description

La fonction torch.load() de PyTorch utilise l'unpickler de Python, capable, au travers du protocole __reduce__, d'instancier des objets arbitraires et d'exécuter du code lors du chargement. Lorsque l'argument weights_only n'est pas positionné à True, un fichier pytorch_model.bin malveillant — issu d'un dépôt HuggingFace compromis ou d'un chemin local piégé — peut déclencher une exécution de code arbitraire (RCE) sans interaction supplémentaire de l'utilisateur.

4.2 Sites concernés

Quatre appels vulnérables ont été identifiés : trois dans le découpage des checkpoints (utils.py) et un dans le chargement d'adaptateurs LoRA (airllm_lora.py) :

Fichier	Ligne (avant)	Rôle
air_llm/airllm/utils.py	≈585	Index d'un checkpoint mono-fichier .bin
air_llm/airllm/utils.py	≈757	Chargement d'un shard par couche (voie multi-shards)
air_llm/airllm/utils.py	≈775	Chargement d'un fichier modèle unique (voie mono-fichier)
air_llm/airllm/airllm_lora.py	≈375	Chargement d'un adaptateur LoRA (load_adapter)

4.3 Impact

Compromission totale de la machine hôte : exécution de commandes arbitraires, exfiltration de données, installation d'une persistance. Il s'agit du risque le plus grave de l'audit, car il est déclenché par la simple ouverture d'un modèle.

4.4 Mesure appliquée

- **Helper central —** ajout de _safe_torch_load() (utils.py) qui force weights_only=True.
- **Couverture complète —** les quatre sites vulnérables sont redirigés vers ce helper.
- **Verrouillage —** suppression du repli « except TypeError » de load_adapter() qui réactivait silencieusement l'unpickler non restreint sur les anciennes versions de torch.

```
def _safe_torch_load(path):
return torch.load(path, map_location='cpu', weights_only=True)
```

Les fichiers de poids ne contenant que des tenseurs et des conteneurs simples, weights_only=True est à la fois suffisant fonctionnellement et indispensable pour la sécurité. La voie principale de la bibliothèque (safetensors) n'est pas affectée, ce format étant sûr par conception.

5. V2 — Exécution de code distant non maîtrisée (ÉLEVÉ)

5.1 Description

L'option trust_remote_code de HuggingFace autorise l'exécution du code Python embarqué dans un dépôt de modèles (fichiers modeling_*.py, configuration_*.py, tokenization_*.py). Cette option était codée en dur à True à plusieurs endroits : dans la détection d'architecture (AutoModel.get_module_class) et dans les chargeurs de tokenizer (airllm_base, MLX, Baichuan). Tout dépôt dont l'identifiant était fourni exécutait ainsi son code distant, sans possibilité de refus pour

l'utilisateur.

5.2 Impact

Exécution de code tiers arbitraire à l'initialisation du modèle. Combinée au téléchargement automatique (snapshot_download), cette option revenait à faire confiance à du code non audité téléchargé à la volée.

5.3 Mesure appliquée

Une politique tri-état a été introduite, cohérente sur l'ensemble des chargeurs :

Valeur	Comportement
None (défaut)	Parseur/tokenizer natif essayé en premier ; repli sur le code distant uniquement en cas d'échec.
True	Fait toujours confiance au code distant.
False	N'exécute jamais de code distant (lève une erreur si le modèle l'exige).

- **Phase 1 —** AutoModel.get_module_class rend le flag configurable (défaut auto).
- **Phase 2 —** généralisation à tous les chargeurs : AirLLMBaseModel, AirLLMLlamaMlx, AirLLMBaichuan et les deux trainers LoRA acceptent trust_remote_code. La valeur résolue est mémorisée sur self.trust_remote_code et réutilisée par get_tokenizer(), garantissant la cohérence entre config et tokenizer.

```
# airllm_base.py — politique tri-état (extrait)
if self.trust_remote_code is False:
config = AutoConfig.from_pretrained(path, trust_remote_code=False, **tok)
elif self.trust_remote_code is True:
config = AutoConfig.from_pretrained(path, trust_remote_code=True, **tok)
else: # auto : natif d'abord, distant en repli
try:
config = AutoConfig.from_pretrained(path, trust_remote_code=False, **tok)
except Exception:
config = AutoConfig.from_pretrained(path, trust_remote_code=True, **tok)
```

6. V3 — Robustesse et suppression de fichiers (MOYEN)

6.1 Description

Dans remove_real_and_linked_file(), la variable targetpath n'était initialisée que dans la branche traitant les liens symboliques. Sur un chemin ordinaire (non symlink), la fonction atteignait l'instruction « if (targetpath) » avec une variable locale non définie, provoquant une UnboundLocalError. Cette fonction est invoquée par l'option delete_original, qui supprime les fichiers de poids originaux de l'utilisateur après découpage.

6.2 Impact

Plantage de la fonction, et suppression destructrice de fichiers sans validation de périmètre de chemin (les chemins étant construits dynamiquement à partir du répertoire de checkpoint).

6.3 Mesure appliquée

- **Correction —** targetpath est initialisé à None ; la comparaison des chemins est normalisée via str() pour fiabiliser la détection des symlinks.
- **Documentation —** le contrat de la fonction (suppression du lien et de sa cible réelle) est explicité.

7. V4 — Intégrité de la chaîne d'approvisionnement (MOYEN)

7.1 Description

Le fichier requirements.txt référençait transformers depuis la branche main (référence mouvante). Une branche ou un tag déplaçable peut être réécrit silencieusement, ce qui brise la reproductibilité des builds et expose à une compromission en amont : une dépendance modifiée côté upstream serait installée sans changement visible côté projet.

7.2 Impact

Builds non reproductibles et risque d'injection de code via une dépendance altérée.

7.3 Mesure appliquée

Les trois dépendances installées depuis git sont désormais épinglées à des SHA de commit immubles, chacun vérifié comme réel via git ls-remote :

Dépendance	SHA épinglé
transformers	28034545c35acac05793695bd5c878f66e6b544e
peft	7120c9a2636f93c8db3fc4ae466e02f338ecead8
accelerate	47b9ac10710dd51e6e4e1304223d6dc543c47a54

Le packaging officiel (setup.py) conserve par ailleurs une plage PyPI saine et bornée (transformers>=4.49,<6) ; seules les broches de développement/CI ont été figées à des commits.

8. Phase 2 — Généralisation tri-état de trust_remote_code

La phase 1 du correctif V2 s'était concentrée sur la détection d'architecture. La phase 2 a généralisé la politique à l'ensemble des chargeurs afin qu'aucun chemin d'exécution ne contourne le contrôle. Les entités suivantes acceptent désormais le paramètre trust_remote_code (défaut None) :

- AirLLMBaseModel.__init__ (chemin d'inférence générique) ;
- AirLLMLlamaMlx.__init__ (chemin macOS / MLX) ;
- AirLLMBaichuan.get_tokenizer (tokenizer personnalisé, respecte un False explicite) ;
- AirLLMLoRA et AirLLMLoRAQwen4Exp (entraînement).

La valeur résolue est mémorisée sur self.trust_remote_code durant l'initialisation (phase config) puis réutilisée par get_tokenizer(), ce qui garantit que le tokenizer et la config suivent une politique cohérente, au lieu que le tokenizer n'exécute le code distant que de façon inconditionnelle. AutoModel.from_pretrained transmet le flag au constructeur du modèle, et get_module_class le lit sans le consommer pour la détection d'architecture.

> Les scripts d'entraînement (training/qlora.py, rlhf/qlora_dpo.py) étaient déjà conformes : leur paramètre trust_remote_code est défini à False par défaut (opt-in). Aucune modification n'a été nécessaire de ce côté.

9. Construction des impacts : matrice avant / après

Le tableau suivant synthétise la réduction de l'exposition obtenue par chaque mesure, ainsi que le risque résiduel après correction.

Risque	Exposition avant	Exposition après	Risque résiduel
V1 RCE pickle	Certaine (code arbitraire)	Nulle (weights_only=True)	Négligeable
V2 Code distant	Systématique (non désactivable)	À la demande (tri-état)	Faible — inhérent à l'usage
V3 Suppression	Plantage + non validée	Robuste + documentée	Faible — confiné au cache HF
V4 Supply chain	Non reproductible	Épinglé (SHA immuables)	Négligeable

10. Volumétrie des changements

L'ensemble des correctifs de sécurité (phases 1 et 2) s'établit comme suit.

Fichier	Δ lignes	Nature du changement
air_llm/airllm/utils.py	+26 / −7	Helper _safe_torch_load (4 sites) + fix remove_real_and_linked_file
air_llm/airllm/airllm_base.py	+37 / −19	trust_remote_code tri-état + get_tokenizer aligné
air_llm/airllm/airllm_llama_mlx.py	+36 / −19	Politique tri-état (config + tokenizer MLX)
air_llm/airllm/airllm_baichuan.py	+6 / −3	Tokenizer respectant la politique résolue
air_llm/airllm/auto_model.py	+46 / −12	Flag configurable, transmis au constructeur, tri-état
air_llm/airllm/airllm_lora.py	+4 / −0	Deux trainers LoRA : paramètre trust_remote_code
air_llm/tests/test_safe_load.py	nouveau (80)	Test anti-RCE pickle (skip si torch absent)
air_llm/tests/test_trust_remote_code.py	nouveau (97)	Test tri-état trust_remote_code (5 cas)

Fichier	Δ lignes	Nature du changement
requirements.txt	+5 / −3	Épinglage git vers SHA immuables

Deux nouveaux fichiers de test et trois fichiers de documentation (rapports .docx/.pdf et leurs générateurs) complètent ce lot.

11. Stratégie de test et validation

11.1 Tests automatisés

Test	Cas couverts	Dépendances
test_safe_load.py	Rejet d'un pickle malveillant sans exécution ; chargement normal (skip si absent)	torch (skip si absent)
test_trust_remote_code.py	5 cas : auto natif, repli custom-arch, True distant-seul, False aucun repli, auto-inconnu (AutoConfig mocké)	aucune (AutoConfig mocké)

Le test tri-état stubbe transformers.AutoConfig avec un enregistreur qui lève une exception lorsque le parseur natif (trust_remote_code=False) est sollicité sur une architecture personnalisée. Il vérifie ainsi, sans téléchargement ni GPU, que : en mode auto le parseur natif est essayé en premier ; le repli distant n'a lieu qu'en cas d'échec ; trust_remote_code=True utilise directement le code distant ; trust_remote_code=False n'exécute jamais de code distant, même en cas d'échec natif.

11.2 Contrôles manuels

Contrôle	Résultat
Compilation (py_compile) de tous les fichiers modifiés	Succès
Recherche de torch.load non sécurisé résiduel	Aucun
Recherche de trust_remote_code=True inconditionnel	Aucun (uniquement dans des branches légitimes)
Vérification des SHA de dépendances (git ls-remote)	Tous réels
Exécution des tests unittest	Succès

12. Risques résiduels et limites assumées

Les points suivants, hors du périmètre des lots corrigés ou intrinsèques à la conception, sont documentés comme risques résiduels :

- **Liens symboliques —** link_or_copy_file() suit les liens symboliques via realpath, par conception : les caches HuggingFace stockent les fichiers en symlinks vers un répertoire de blobs. Un confinement strict casserait ce mécanisme. L'impact reste faible, le contenu demeurant dans le cache HF.
- **Compatibilité .bin anciens —** les checkpoints .bin très anciens contenant autre chose que des tenseurs échoueront désormais explicitement avec weights_only=True. C'est le comportement voulu (sécurité avant permissivité) ; la voie safetensors n'est pas affectée.
- **Modèles exigeant le code distant —** en mode False, un modèle dont le parseur natif échoue lèvera une erreur. C'est le choix assumé de l'utilisateur ; le mode auto (None) reste le défaut confortable.

13. Conclusion et recommandations

La construction des impacts démontre une réduction substantielle de la surface d'attaque. La vulnérabilité critique d'exécution de code arbitraire est intégralement neutralisée ; l'exécution de code distant passe sous contrôle explicite et tri-état de l'utilisateur ; la robustesse des opérations sur fichiers est rétablie ; et la chaîne d'approvisionnement est rendue reproductible. L'ensemble est couvert par des tests automatisés et versionné.

Recommandations pour la suite :

- **CI —** exécuter la suite unittest sur chaque pull request, et ajouter un lint de sécurité (ex. Bandit) pour détecter tout nouveau torch.load/pickle non restreint.
- **Documentation —** mentionner le paramètre trust_remote_code dans le README, avec la recommandation de le laisser à None (auto) sauf besoin précis.
- **Veille —** maintenir les SHA épinglés à jour via des mises à jour régulières et revues.

Annexe A — Détail des commits

SHA	Message	Contenu principal
a6d631f	security: harden checkpoint loading and dependency pinning	V1 (RCE pickle), V4 (épinglage), fix V3 + test_safe_load
6b97abf	docs: add security impact-study Word report and its generator	Rapport .docx + gen_security_report.py
eb9a06f	docs: add PDF version of the security impact-study report	Rapport .pdf + gen_security_report_pdf.py
ac9e260	security: make trust_remote_code tri-state across all phase-2 loaders	V2 phase 2 (loaders) + test_trust_remote_code

Annexe B — Inventaire des fichiers audités

46 fichiers Python répartis comme suit :

Répertoire	Nb fichiers	Rôle
air_llm/airllm/	18	Cœur de la bibliothèque (modèles, utils, persistance, LoRA)
air_llm/tests/	9	Tests unitaires
air_llm/ + examples	3	Point d'entrée, exemple d'inférence, scripts d'exemple
anima_100k/	2	Modèle et script d'entraînement long contexte
training/ + rlhf/ + scripts	3	Scripts QLoRA / DPO et utilitaires

Annexe C — Références des correctifs (extraits de code)

C.1 safe torch load (utils.py)

```
def _safe_torch_load(path):
return torch.load(path, map_location='cpu', weights_only=True)
```

C.2 remove real and linked file (utils.py)

```
targetpath = None
real = os.path.realpath(str(to_delete))
if real != str(to_delete):
targetpath = real
os.remove(to_delete)
if targetpath:
os.remove(targetpath)
```

C.3 Épinglage des dépendances (requirements.txt)

```
transformers @ git+https://github.com/huggingface/transformers.git@28034545...
peft @ git+https://github.com/huggingface/peft.git@7120c9a2...
accelerate @ git+https://github.com/huggingface/accelerate.git@47b9ac10...
```

Annexe D — Glossaire

Terme	Définition
RCE	Remote Code Execution — exécution de code arbitraire à distance sur la machine hôte.
pickle	Module de sérialisation Python, capable d'exécuter du code à la désérialisation.
weights_only	Option de torch.load restreignant le chargement aux tenseurs (bloque le code arbitraire).
trust_remote_code	Option HuggingFace autorisant l'exécution du code Python embarqué dans un dépôt de modèles.

Terme	Définition
safetensors	Format de poids sûr (mappé mémoire, sans exécution de code).
Supply chain	Chaîne d'approvisionnement logicielle ; ici, l'intégrité des dépendances installées.
SHA	Empreinte immubable identifiant un commit git de façon déterministe.